

СОДЕРЖАНИЕ

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ	2
ВВЕДЕНИЕ	3
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ЗАДАЧИ	6
1.1 Адаптивное языковое обучение: рынок и аудитория	6
1.2 Сравнительный анализ существующих решений	9
1.3 Подходы к генерации упражнений	13
1.4 Постановка задачи разработки системы генерации	15
2 СРАВНЕНИЕ МЕТОДОВ ГЕНЕРАЦИИ УПРАЖНЕНИЙ	19
2.1 Проектирование экспериментов	19
2.2 Разведочный анализ датасета EXPEST	20
2.3 Подготовка данных	25
2.4 Выбор и обоснование моделей генерации упражнений	27
2.5 Дообучение моделей	29
2.6 Экспериментальная оценка качества методов	32
2.7 Обоснование эффективности предложенного решения	35
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	38
Приложение А	40
Приложение Б	43
Приложение В	45
Приложение Г	48

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

ВВЕДЕНИЕ

Персонализация обучения является одним из ключевых трендов современного онлайн-образования. Глобальный рынок цифрового изучения языков демонстрирует устойчивый рост, а такие платформы, как Duolingo, ELSA Speak и Mondly, привлекают сотни миллионов пользователей. Интеграция технологий искусственного интеллекта в образовательный процесс открывает новые возможности для масштабируемой персонализации обучения [1]. Исследования показывают, что ИИ-инструменты способны обеспечивать мгновенную обратную связь, адаптировать содержание заданий под индивидуальный уровень учащегося и значительно повышать учебные результаты по сравнению с традиционными методами [2].

Вместе с тем традиционный подход, при котором все ученики получают одинаковые задания вне зависимости от индивидуального уровня подготовки и типа допускаемых ошибок, демонстрирует свою неэффективность. Ручное создание персонализированных упражнений представляет собой трудоёмкий процесс, который не масштабируется при большом количестве учащихся [3]. При этом стоимость занятий с репетитором остаётся высокой и недоступной для значительной части целевой аудитории, что ограничивает возможности индивидуальной языковой практики.

Существующие автоматизированные решения, как правило, основаны на обращениях к облачным API крупных языковых моделей, что влечёт за собой высокую стоимость при массовом использовании, зависимость от внешних сервисов и потенциальные риски, связанные с приватностью данных учеников. Альтернативные шаблонные методы генерации упражнений обеспечивают предсказуемость результата, но ограничены единственным типом задания и не способны адаптироваться под конкретный тип ошибки. За последние два десятилетия развития искусственного интеллекта в образовании исследователи установили, что наиболее активно развивающимися направлениями стали интеллектуальные обучающие системы, методы обработки естественного языка для языкового обучения и образовательные роботы [4]. Однако единая сквозная система, автоматически связывающая диагностику ошибок ученика с генерацией релевантных упражнений без зависимости от внешних облачных сервисов, в настоящее время отсутствует.

Практическая значимость настоящей работы определяется несколькими факторами. Во-первых, разрабатываемая система позволяет

повысить эффективность обучения за счёт автоматической генерации упражнений, нацеленных на конкретные грамматические ошибки ученика. Во-вторых, использование компактных open-source моделей, развёртываемых локально, существенно снижает операционные расходы на генерацию контента по сравнению с облачными решениями. В-третьих, автоматическая генерация обеспечивает ученикам мгновенную обратную связь – упражнения создаются в реальном времени непосредственно после анализа допущенных ошибок. Система разрабатывается в рамках платформы изучения иностранных языков «SAYIT» и предоставляет интеграционный интерфейс для встраивания в образовательную среду.

Целью работы является разработка автономной системы генерации персонализированных грамматических упражнений по английскому языку с применением методов искусственного интеллекта, не зависящей от внешних облачных сервисов и пригодной для интеграции в образовательную платформу.

Для достижения поставленной цели необходимо решить ряд следующих задач:

- 1) Провести анализ предметной области: обзор рынка адаптивного языкового обучения, исследование целевой аудитории, сравнительный анализ существующих решений.
- 2) Исследовать и сравнить методы генерации языковых упражнений, выполнить экспериментальную оценку качества и обосновать выбор подхода.
- 3) Спроектировать и реализовать систему генерации персонализированных упражнений с интеграционным интерфейсом для платформы «SAYIT».
- 4) Выполнить тестирование системы и подготовить оценку производительности и ресурсоёмкости.
- 5) Провести пилотную апробацию на целевой аудитории, оценить образовательный эффект и собрать экспертную обратную связь.
- 6) Оценить экономическую эффективность решения и разработать план масштабирования.

Объектом исследования является процесс автоматической генерации персонализированных языковых упражнений. Предметом исследования выступают методы генерации грамматических упражнений на основе дообученных языковых моделей и шаблонных алгоритмов.

Научная новизна работы состоит в разработке сквозного конвейера реактивной персонализации: система принимает от образовательной платформы структурированные данные об ошибке ученика и генерирует упражнение, таргетированное на конкретный тип ошибки в конкретном контексте.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ЗАДАЧИ

1.1 Адаптивное языковое обучение: рынок и аудитория

Целевым рынком для системы генерации персонализированных упражнений с применением методов искусственного интеллекта является сегмент онлайн-изучения иностранных языков в составе глобального рынка образовательных технологий (EdTech).

Рынок EdTech демонстрирует устойчивый рост, чему способствуют цифровизация образования, развитие мобильных технологий и растущий интерес к обучению вне традиционных институтов. Особенно быстро развивается сегмент онлайн-изучения языков: он привлекает как частных пользователей, стремящихся к карьерному росту и академической мобильности, так и корпоративных клиентов, заинтересованных в повышении языковой компетенции сотрудников. По оценкам Grand View Research, глобальный рынок онлайн-изучения языков растёт со среднегодовым темпом около 17%, при этом сегмент мобильных приложений для самостоятельного обучения занимает более 60% выручки [5]. Основными драйверами роста остаются доступность, гибкость формата и возможность обучения в индивидуальном темпе.

Ключевым трендом стала персонализация обучения с помощью искусственного интеллекта. Современные платформы используют ИИ для адаптации контента под уровень и стиль усвоения конкретного пользователя, автоматической оценки произношения и построения индивидуальных траекторий обучения. Большие языковые модели позволяют создавать реалистичные диалоги, имитирующие разговор с носителем языка, а анализ поведенческих данных помогает рекомендовать темы и темп занятий. Экспериментальные исследования подтверждают, что ИИ-инструменты способны значительно повышать учебные результаты по сравнению с традиционными методами за счёт мгновенной обратной связи и адаптации заданий [2].

Для понимания текущего состояния рынка рассмотрим подходы к персонализации, реализованные в наиболее популярных платформах.

Duolingo – крупнейшая геймифицированная платформа для изучения более 40 языков через короткие интерактивные уроки. Платформа использует собственную ИИ-модель Birdbrain для персонализации: система анализирует сильные и слабые стороны каждого пользователя, предсказывает вероятность забывания изученного материала и подбирает из пула доступных упражнений те, которые наиболее релевантны для конкретного ученика в данный момент

[6]. С недавнего времени Duolingo применяет большие языковые модели для генерации новых упражнений: дизайнер задаёт параметры (язык, уровень CEFR, грамматический фокус), после чего модель создаёт набор заданий за несколько секунд [7]. Однако сгенерированные упражнения проходят обязательную ручную модерацию перед публикацией и формируют статический пул, а не создаются реактивно в ответ на конкретную ошибку ученика. Кроме того, платформа ориентирована преимущественно на базовый и средний уровни владения языком, а расширенные ИИ-функции (объяснение ошибок, ролевые диалоги) доступны только в рамках платной подписки Duolingo Max.

ELSA Speak – ИИ-приложение, специализирующееся на улучшении произношения английского языка. Платформа использует собственную технологию распознавания речи для анализа произношения на уровне отдельных фонем, оценки интонации и ритма. На основе результатов входного тестирования система формирует персонализированный план обучения, который адаптируется по мере прогресса пользователя [8]. ELSA также предлагает AI-ролевые игры для практики разговорных ситуаций с мгновенной обратной связью по произношению, грамматике и беглости. Ограничения платформы связаны с узкой специализацией: основной фокус – произношение и беглость речи, тогда как целенаправленная работа над грамматическими ошибками, генерация письменных упражнений и адаптация заданий под конкретный тип грамматической ошибки выходят за рамки её функциональности.

Babbel – платформа, предлагающая структурированные курсы, разработанные профессиональными лингвистами и методистами. В отличие от геймифицированного подхода Duolingo, Babbel делает акцент на грамматических объяснениях, контекстных диалогах и практических сценариях повседневного общения. Персонализация реализована на уровне адаптивного подбора упражнений из готового контента: система учитывает результаты предыдущих заданий и предлагает повторение материала по алгоритму интервального повторения. Однако все упражнения создаются вручную – платформа не генерирует новый контент под конкретные ошибки ученика, а лишь выбирает наиболее подходящие задания из заранее подготовленного пула. Набор поддерживаемых языков ограничен, а формат заданий остаётся фиксированным вне зависимости от типа допускаемых ошибок.

Таким образом, существующие платформы реализуют персонализацию преимущественно на уровне подбора заданий из готового

контента или адаптации сложности. Генерация принципиально нового упражнения, таргетированного на конкретную ошибку конкретного ученика в реальном времени, остаётся нерешённой задачей для большинства коммерческих решений. При этом ручное создание персонализированных упражнений представляет собой трудоёмкий процесс, который не масштабируется при большом количестве учащихся [3].

Целевая аудитория системы генерации персонализированных упражнений включает несколько ключевых сегментов, каждый из которых сталкивается с характерными проблемами.

Студенты и соискатели, готовящиеся к международным языковым экзаменам (IELTS, TOEFL и др.), испытывают дефицит доступной разговорной и грамматической практики. Стоимость занятий с репетитором-носителем языка остаётся высокой, а при потребности в ежедневных тренировках затраты становятся непосильными для большинства учащихся. При этом бесплатные тренажёры, основанные на выборе вариантов ответа, не формируют продуктивных навыков: пользователи могут тратить значительное время в приложениях, но на реальном экзамене получать низкий балл, поскольку механический навык выбора правильного ответа не конвертируется в навык свободного использования языковых конструкций.

Школьники и студенты, выполняющие домашние задания по иностранному языку, сталкиваются с разрывом цикла обратной связи. Ожидание проверки работы преподавателем занимает от одного до нескольких дней, к моменту получения исправлений ученик уже утрачивает контекст ошибки, и одни и те же ошибки повторяются на протяжении длительного времени. Родители вынуждены обращаться к репетиторам не столько для объяснения новых тем, сколько для оперативной проверки и разбора заданий, что воспринимается как неоправданная переплата.

Общей проблемой для всех сегментов является отсутствие автоматической связи между допущенной ошибкой и предлагаемым упражнением. Существующие платформы либо предлагают одинаковые задания всем пользователям, либо адаптируют общую сложность, но не тип и содержание задания под конкретную грамматическую ошибку в конкретном контексте.

Результаты масштабных опросов подтверждают выявленные проблемы. Согласно исследованию European School Education Platform, главной проблемой в изучении языков респонденты назвали ограниченную

экспозицию к целевому языку за пределами класса (53%). Далее следуют нехватка возможностей для аутентичной языковой практики (44%), низкая мотивация учащихся (42%) и ограниченное учебное время (41%). Недостаточную поддержку индивидуальных потребностей в обучении и неэффективные методики преподавания отметили по 23% опрошенных. При этом интеграцию технологий и персонализированных траекторий обучения как перспективное направление поддержали 27% респондентов [9]. Эти данные свидетельствуют о системном разрыве между потребностью учащихся в индивидуализированной практике и возможностями существующих образовательных решений – разрыве, который может быть сокращён за счёт автоматической генерации упражнений, адаптированных под конкретные ошибки ученика.

1.2 Сравнительный анализ существующих решений

Сравнительный анализ существующих решений проводится в два этапа. На первом этапе выполняется анализ конкурентных технических решений – он позволяет количественно оценить разрабатываемую систему относительно ближайших аналогов по набору технических и экономических критериев. На втором этапе проводится SWOT-анализ, необходимый для выявления внутренних сильных и слабых сторон проекта, а также внешних возможностей и угроз, определяющих перспективы его развития.

Анализ конкурентных технических решений. Для оценки конкурентоспособности разрабатываемой системы генерации персонализированных упражнений проведён сравнительный анализ с тремя наиболее близкими решениями на рынке:

- Duolingo (к1) – крупнейшая платформа с AI-генерацией упражнений и адаптивным подбором заданий;
- ELSA Speak (к2) – ИИ-приложение с персонализированным обучением, фокус на произношении;
- Babbel (к3) – платформа со структурированными курсами и адаптивным повторением из готового пула.

Анализ конкурентных решений определяется по формуле (3):

$$K = \sum B_i \cdot B_{i'}, \#(3)$$

где K – конкурентоспособность научной разработки или конкурента; B_i – вес показателя (в долях единицы); $B_{i'}$ – балл i -го показателя.

Сравнение технических и экономических характеристик данных продуктов с разрабатываемым решением представлено в таблице 1.1.

Таблица 1.1 – Оценочная карта сравнения конкурентных технических решений

Критерии оценки	Вес	Б _ф	Б _{к1}	Б _{к2}	Б _{к3}	К _ф	К _{к1}	К _{к2}	К _{к3}
Технические критерии оценки									
1. Адаптация упражнений под тип ошибки	0.12	5	3	2	3	0.60	0.36	0.24	0.36
2. Генерация нового контента (не из пула)	0.12	5	4	1	1	0.60	0.48	0.12	0.12
3. Разнообразие типов заданий	0.08	5	4	2	4	0.40	0.32	0.16	0.32
4. Работа без облачных API	0.1	5	1	1	1	0.50	0.10	0.10	0.10
5. Мгновенная обратная связь	0.08	5	5	4	4	0.40	0.40	0.32	0.32
6. Глубина грамматического покрытия	0.08	4	3	2	4	0.32	0.24	0.16	0.32
7. Интеграция через API	0.07	5	2	2	3	0.35	0.14	0.14	0.21
Экономические критерии оценки									
1. Стоимость эксплуатации при масштабировании	0.1	5	2	2	2	0.50	0.20	0.20	0.20
2. Независимость от внешних сервисов	0.08	5	1	1	1	0.40	0.08	0.08	0.08
3. Открытость технологии (open-source)	0.07	5	1	1	1	0.35	0.07	0.07	0.07
4. Готовность к встраиванию в платформу	0.05	5	2	2	3	0.25	0.10	0.10	0.15
5. Зрелость и размер пользовательской базы	0.05	2	5	4	4	0.10	0.25	0.20	0.20
Итого	1.00	56	33	24	31	4.77	2.74	1.89	2.45

Результаты оценки показывают, что разрабатываемая система ($K_{\text{ф}} = 4.77$) превосходит все рассмотренные аналоги по совокупности критериев. Основное конкурентное преимущество заключается в способности генерировать упражнения реактивно, под конкретную ошибку конкретного ученика, без обращения к внешним облачным сервисам. Duolingo ($K_{\text{к1}} = 2.74$) ближе всего к разрабатываемому решению благодаря использованию LLM для генерации контента, однако его генерация направлена на пополнение статического пула упражнений с обязательной ручной модерацией, а не на реактивную персонализацию. ELSA Speak ($K_{\text{к2}} = 1.89$) и Babbel ($K_{\text{к3}} = 2.45$) не генерируют новый контент, ограничиваясь подбором из заранее подготовленных заданий.

Слабой стороной разрабатываемого решения является отсутствие собственной пользовательской базы и зрелости продукта – это закономерно для системы на стадии разработки и может быть компенсировано за счёт интеграции с платформой «SAYIT».

SWOT-анализ. Для комплексной оценки внутренних и внешних факторов проекта проведён SWOT-анализ. В таблице 1.2 приведены результаты первого этапа SWOT-анализа.

Таблица 1.2 – Матрица SWOT-анализа

	Сильные стороны: С1. Реактивная генерация упражнений под конкретную ошибку ученика С2. Автономность: работа без облачных API С3. Разнообразие форматов заданий (4 типа) С4. Открытость технологии (open-source модели) С5. Низкая стоимость эксплуатации	Слабые стороны: Сл1. Отсутствие собственной пользовательской базы Сл2. Ограниченность одним языком (английский) Сл3. Зависимость от качества входных данных детектора ошибок Сл4. Необходимость GPU для инференса
Угрозы: У1. Появление аналогичных функций у крупных платформ (Duolingo, ELSA) У2. Быстрое развитие облачных LLM, снижающее преимущество автономности У3. Ограниченность обучающих датасетов для новых языков У4. Риски качества генерации (галлюцинации модели)		
Возможности: В1. Растущий спрос на персонализацию в EdTech В2. Развитие компактных open-source LLM В3. Интеграция с другими образовательными платформами через API В4. Расширение на другие языки и типы ошибок		

Для определения соответствия сильных и слабых сторон внешним условиям построены интерактивные матрицы проекта (таблицы 1.3–1.6). В каждой ячейке матрицы будет либо символ «+» (сильное соответствие), либо «-», (слабое соответствие), либо «0» (если есть неопределенность между первыми двумя вариантами).

Таблица 1.3 – Интерактивная матрица: сильные стороны и возможности

	C1	C2	C3	C4	C5
B1	+	-	+	-	+
B2	+	+	0	+	0
B3	+	+	+	0	+
B4	+	-	+	0	0

Таблица 1.4 – Интерактивная матрица: сильные стороны и угрозы

	C1	C2	C3	C4	C5
У1	+	-	+	+	+
У2	-	-	0	0	-
У3	-	0	-	0	0
У4	0	0	+	0	0

Таблица 1.5 – Интерактивная матрица: слабые стороны и возможности

	Сл1	Сл2	Сл3	Сл4
В1	-	-	0	-
В2	0	0	0	+
В3	+	-	0	-
В4	-	+	-	0

Таблица 1.6 – Интерактивная матрица: слабые стороны и угрозы

	Сл1	Сл2	Сл3	Сл4
У1	+	+	0	0
У2	0	0	0	+
У3	-	+	+	0
У4	+	0	+	0

В результате анализа интерактивных матриц выделены следующие ключевые корреляции:

- В1В3С1С3С5: растущий спрос на персонализацию и возможность интеграции через API усиливают ключевое преимущество системы – реактивную генерацию и низкую стоимость;
- У1С1С3С4С5: появление аналогичных функций у крупных платформ компенсируется автономностью, открытостью технологии и экономической эффективностью;
- В4Сл2: расширение на другие языки ограничено текущей поддержкой только английского языка;
- У1У4Сл1Сл3: отсутствие пользовательской базы и зависимость от качества детектора ошибок создают уязвимость при появлении конкурентов и рисках галлюцинаций модели.

В заключение данного этапа составляется итоговая матрица SWOT-анализа, представленная в таблице 1.7.

Таблица 1.7 – Итоговая матрица SWOT-анализа

	Сильные стороны: С1. Реактивная генерация упражнений под конкретную ошибку ученика	Слабые стороны: Сл1. Отсутствие собственной пользовательской базы Сл2. Ограниченность одним языком (английский)
--	--	--

	C2. Автономность: работа без облачных API C3. Разнообразие форматов заданий (4 типа) C4. Открытость технологии (open-source модели) C5. Низкая стоимость эксплуатации	Сл3. Зависимость от качества входных данных детектора ошибок Сл4. Необходимость GPU для инференса
Угрозы: У1. Появление аналогичных функций у крупных платформ (Duolingo, ELSA) У2. Быстрое развитие облачных LLM, снижающее преимущество автономности У3. Ограниченность обучающих датасетов для новых языков У4. Риски качества генерации (галлюцинации модели)	Реактивная генерация, разнообразие форматов, открытость технологии и низкая стоимость эксплуатации позволяют сохранить конкурентное преимущество даже при появлении аналогичных функций у крупных платформ. Структурная валидация генерируемых упражнений снижает риски галлюцинаций модели.	Отсутствие пользовательской базы и зависимость от качества детектора ошибок усиливают уязвимость при появлении конкурентов и рисках галлюцинаций. Необходимость GPU ограничивает возможности масштабирования на фоне удешевления облачных LLM. Ограниченность одним языком в сочетании с дефицитом датасетов затрудняет расширение языкового покрытия.
Возможности: В1. Растущий спрос на персонализацию в EdTech В2. Развитие компактных open-source LLM В3. Интеграция с другими образовательными платформами через API В4. Расширение на другие языки и типы ошибок	Растущий спрос на персонализацию в EdTech и возможность интеграции через API усиливают ключевые преимущества системы – реактивную генерацию, разнообразие форматов и низкую стоимость. Развитие компактных open-source LLM укрепляет автономность и открытость технологии, расширяя возможности локального развёртывания.	Интеграция с образовательными платформами через API компенсирует отсутствие собственной пользовательской базы, обеспечивая доступ к аудитории. Расширение на другие языки позволяет преодолеть текущую ограниченность английским. Развитие компактных LLM снижает требования к GPU, упрощая развёртывание.

Проведённый анализ показывает, что разрабатываемая система обладает существенным конкурентным преимуществом в виде реактивной персонализации и автономности. Основные риски связаны с ограниченностью языкового покрытия и зрелостью продукта, что может быть смягчено за счёт поэтапного расширения функциональности и интеграции с платформой SAYIT, обеспечивающей доступ к целевой аудитории.

1.3 Подходы к генерации упражнений

Задача автоматической генерации языковых упражнений может быть решена двумя принципиально различными подходами: шаблонным и нейросетевым. Шаблонные методы используют заранее определённые правила и алгоритмы для преобразования входного текста в задание фиксированного формата. Нейросетевые методы, напротив, опираются на языковые модели, способные генерировать упражнения произвольной структуры на основе обученных представлений о языке.

Шаблонные методы генерации. Шаблонные (rule-based) подходы к генерации упражнений имеют длительную историю в области автоматизированного обучения языкам. Их суть заключается в применении детерминированных правил к входному тексту: алгоритм идентифицирует целевой языковой элемент, извлекает его из предложения и формирует задание по заранее заданному шаблону. Основным достоинством таких методов является предсказуемость и структурная корректность результата: каждое сгенерированное упражнение гарантированно соответствует заданному формату и содержит правильный ответ. Кроме того, шаблонные методы не требуют обучающих данных и вычислительных ресурсов для инференса.

Ограничения шаблонных подходов связаны с их жёсткостью: каждый шаблон порождает ровно один тип задания, а адаптация под контекст ошибки или уровень ученика требует написания дополнительных правил вручную. Объём генерируемого контента, как правило, ограничен одним заданием на входное предложение, а разнообразие форматов определяется количеством реализованных шаблонов.

Среди шаблонных методов, применимых к задаче генерации грамматических упражнений, стоит выделить два подхода – Fill-in-the-Blanks и Sentence Reconstruction.

Fill-in-the-Blanks – метод генерации упражнений на заполнение пропуска. Алгоритм принимает на вход пару «ошибочное предложение – исправленное предложение», определяет позицию коррекции и извлекает целевое слово из исправленного варианта. Это слово заменяется пропуском, а для формирования вариантов ответа (дистракторов) используется морфологический анализ: определяется часть речи целевого слова, после чего из пула ошибок выбираются слова той же грамматической категории [10]. В результате каждое упражнение содержит одно задание с тремя-четырьмя вариантами ответа, один из которых является правильным. Метод покрывает большинство примеров датасета, однако генерирует только один тип задания и не учитывает конкретный тип грамматической ошибки ученика. Программная реализация метода приведена в Приложении А.

Sentence Reconstruction – метод генерации упражнений на восстановление порядка слов [11]. Алгоритм берёт токены исправленного предложения и перемешивает их в случайном порядке. Задача ученика – восстановить правильную последовательность слов. Метод поддерживает три уровня сложности: на лёгком уровне первое и последнее слова остаются на

месте, перемешивается только середина; на среднем и сложном уровнях перемешиваются все токены. Детали программной реализации представлены в Приложении Б.

Нейросетевые методы генерации. Альтернативный подход к генерации упражнений основан на использовании языковых моделей, обученных генерировать текст на основе контекста. В отличие от шаблонных методов, нейросетевые модели способны генерировать задания произвольной структуры и объёма, адаптируя содержание под тип ошибки, контекст предложения и требуемый формат упражнения [3]. Современные легкие языковые модели с числом параметров до нескольких миллиардов достигают качества, достаточного для практического применения в образовательных системах, при этом допускают локальное развёртывание без обращения к внешним облачным сервисам.

Практическое применение таких моделей в образовательном контексте предполагает формирование входного запроса (промпта), содержащего описание роли генератора, исходное предложение с ошибкой, исправленный вариант, тип ошибки и требуемый формат выходных данных. На выходе модель формирует структурированный ответ – как правило, в формате JSON – включающий набор заданий различных типов: текст вопроса, варианты ответа и правильный ответ. Таким образом, из одной входной пары «ошибка – коррекция» нейросетевой подход способен создать многоформатное упражнение, тогда как шаблонный метод ограничен одним заданием фиксированного типа.

Ключевой техникой, делающей дообучение крупных моделей практически осуществимым на ограниченных вычислительных ресурсах, является LoRA (Low-Rank Adaptation) – метод параметрически эффективной адаптации, при котором обучаются не все веса модели, а лишь низкоранговые матрицы, добавляемые к определённым слоям. Это позволяет адаптировать модель с миллиардами параметров, обновляя при этом лишь несколько процентов от общего числа весов [12]. Благодаря LoRA дообучение становится доступным даже на потребительских GPU, что особенно важно для образовательных проектов с ограниченным бюджетом.

Процесс дообучения предполагает подготовку размеченного датасета, содержащего примеры упражнений в формате структурированного диалога (chat-формат). Разметка обучающих данных, как правило, охватывает несколько типов заданий – например, выбор грамматически правильного варианта, трансформацию предложения, заполнение пропуска с акцентом на

лексику или сопоставление функциональных элементов. Модель, дообученная на таком датасете, обучается воспроизводить заданную структуру ответа и генерировать содержательно релевантные задания для конкретного типа ошибки.

1.4 Постановка задачи разработки системы генерации

Формализация задачи. На основании проведённого анализа предметной области задача настоящей работы формулируется следующим образом: разработать автономную систему генерации персонализированных грамматических упражнений по английскому языку, принимающую на вход структурированные данные об ошибке ученика и формирующую на выходе многоформатное упражнение, готовое к интеграции в платформу SAYIT.

Входные данные системы представляют собой контракт, передаваемый платформой SAYIT, и включают: идентификатор ученика (`user_id`), текст сообщения с ошибкой (`message_content`), детектированную ошибку и её исправление (`grammar_error`), тип ошибки (`error_category`), а также уровень уверенности детектора (`llm_confidence`).

Выходные данные представляют собой структурированный JSON-объект, содержащий от 8 до 14 заданий (`items`) четырёх типов: выбор грамматически правильного варианта (`grammar choice`), трансформация предложения (`transformation`), заполнение пропуска с акцентом на лексику (`vocabulary fill`) и сопоставление функциональных элементов (`functional matching`). Каждое задание включает текст вопроса, варианты ответа и правильный ответ. Тип и содержание заданий автоматически адаптируются под конкретную категорию ошибки ученика, что обеспечивает подстраивание сложности и тематики упражнения под индивидуальный контекст обучения.

Научно-технологическая новизна подхода. Предлагаемая система реализует модель реактивной персонализации: в ответ на конкретную ошибку в конкретном сообщении ученика система в реальном времени генерирует таргетированное упражнение заданного типа, привязанное к контексту этой ошибки. Это создаёт принципиально новое качество образовательного опыта по сравнению со статическими банками заданий, не связанными с индивидуальным контекстом обучения. Новизна состоит в разработке сквозного конвейера, в котором ошибка ученика автоматически поступает на детекцию, после чего результат детекции передаётся генеративной модели, формирующей персонализированное упражнение – весь процесс происходит без ручного вмешательства и без обращения к внешним облачным сервисам. Подобная автоматизированная цепочка, связывающая диагностику ошибок с

генерацией релевантных упражнений в рамках единой локально развёртываемой системы, отсутствует в существующих решениях на рынке EdTech.

Функциональные требования к системе. Система должна обеспечивать генерацию упражнений по произвольному типу грамматической ошибки, представленному в датасете. В рамках одного упражнения должны поддерживаться четыре формата заданий: выбор грамматически правильного варианта, трансформация предложения, заполнение пропуска с акцентом на лексику и сопоставление функциональных элементов. Доля структурно валидных выходных JSON-ответов должна составлять не менее 95% генераций. Для интеграции с платформой SAYIT система должна предоставлять REST API, принимающий структурированный запрос с данными об ошибке ученика и возвращающий готовое упражнение.

Нефункциональные требования. Система должна допускать полностью локальное развёртывание без обращения к внешним облачным API, что исключает зависимость от сторонних сервисов и снижает операционные расходы. В качестве генеративного ядра должны использоваться компактные open-source модели с числом параметров до 4 млрд, допускающие инференс на потребительском GPU. Воспроизводимость результатов обеспечивается фиксацией версий зависимостей и Docker-контейнеризацией среды выполнения.

Сравниваемые методы. Для решения поставленной задачи исследуются два класса подходов. Первый – шаблонные бейзлайны: Fill-in-the-Blanks, основанный на генерации задания с пропуском и дистракторами на основе POS-анализа, и Sentence Reconstruction, предполагающий восстановление порядка слов в перемешанном предложении. Второй – дообученные языковые модели малого размера, адаптированные методом LoRA на размеченном подмножестве датасета.

Метрики оценки. Качество генерации должно оцениваться по двум группам метрик. Первая группа – семантические метрики, позволяющие оценить содержательную близость сгенерированного упражнения к эталонному: насколько текст заданий осмыслен, релевантен исходной ошибке и соответствует ожидаемому уровню языковой сложности. Вторая группа – структурные метрики, отражающие формальную корректность выхода модели: валидность формата ответа, соответствие запрошенному типу задания, полноту обязательных полей и соблюдение заданного диапазона

количества заданий в упражнении. Совместное использование двух групп метрик необходимо для того, чтобы оценить генерацию как с точки зрения смыслового качества контента, так и с точки зрения пригодности результата к автоматической интеграции в образовательную платформу без ручной доработки.

Гипотеза исследования. Дообученная компактная языковая модель с числом параметров до 4 млрд способна автоматически генерировать персонализированные грамматические упражнения по английскому языку, качество которых сопоставимо или превосходит шаблонные методы генерации по совокупности семантических и структурных метрик, при этом не требуя обращения к внешним облачным API. Данная гипотеза объединяет три связанных утверждения: нейросетевая генерация может заменить шаблонное создание упражнений по объёму и разнообразию контента; персонализация на основе конкретных ошибок ученика повышает релевантность заданий по сравнению с универсальными упражнениями; всё это реализуемо автономно, на компактных open-source моделях с локальным развёртыванием.

2 СРАВНЕНИЕ МЕТОДОВ ГЕНЕРАЦИИ УПРАЖНЕНИЙ

2.1 Проектирование экспериментов

Целью экспериментальной части работы является сравнение нейросетевых и шаблонных методов генерации грамматических упражнений на основе ошибок учеников. Для обеспечения корректности сравнения на этапе проектирования фиксируются единая оценочная выборка, набор метрик качества, формат выходных данных и бейзлайн-модели.

Эксперимент предполагает выполнение следующих работ: формирование обучающего набора из датасета грамматических ошибок, дообучение нескольких компактных языковых моделей, реализация шаблонных бейзлайнов на исходных данных датасета, а также оценка всех методов на единой валидационной выборке по набору метрик качества. Разбиение данных на обучающую и валидационную части выполняется в пропорции 90/10 со стратификацией по типу задания, что обеспечивает представленность каждого формата упражнений в оценочном наборе.

Для оценки методов предусматриваются три группы метрик, каждая из которых измеряет отдельный аспект качества генерации:

- BERTScore F1 – оценивает семантическое сходство сгенерированного упражнения с эталонным ответом на основе косинусного сходства контекстных эмбедингов токенов. Для обработки длинных ответов применяется чанковый метод с последующим усреднением. Диапазон значений – от 0 до 1, выше лучше. Метрика применяется только к нейросетевым моделям, поскольку шаблонные бейзлайны генерируют упражнения принципиально иного формата.

- Perplexity – измеряет уверенность модели при порождении текста на валидационном наборе (1). Более низкое значение указывает на лучшее соответствие модели распределению данных. Применяется только к нейросетевым моделям.

$$PPL = \exp \exp(\bar{L}), \#(1)$$

где $\bar{L}$ – средний кросс-энтропийный loss по всем токенам валидационной выборки.

- Task-Relevance Composite (TR) – составная метрика, специально разработанная для данной задачи (2). Оценивает структурную корректность и педагогическую релевантность сгенерированного упражнения. Представляет собой среднее арифметическое пяти подметрик: JSON Valid (доля ответов, успешно парсящихся как валидный JSON), Task Match (доля ответов с корректным типом задания), Error Category Present (доля ответов с

заполненной категорией ошибки), Items Count OK (доля ответов с допустимым числом элементов для данного типа задания) и Answers OK (доля ответов, в которых каждый элемент содержит ответ).

$$TR = \frac{JSON_{valid} + Task_{match} + Items_{ok} + Answer_{ok} + Error_{cat}}{5} \#(2)$$

Диапазон – от 0 до 1, выше лучше. В отличие от двух предыдущих метрик, TR применяется ко всем методам, что позволяет проводить сквозное сравнение нейросетевых и шаблонных подходов.

В качестве бейзлайнов используются два шаблонных метода – Fill-in-the-Blanks и Sentence Reconstruction. Они не требуют обучения нейросетей, служат нижней границей качества и позволяют оценить прирост от применения дообученных моделей. Оба бейзлайна гарантируют детерминированность вывода и не требуют вычислительных ресурсов, однако ограничены одним типом задания, одним элементом на упражнение и не способны адаптироваться к типу ошибки ученика.

В качестве нейросетевых методов выбираются несколько открытых языковых моделей малого размера. Выбор обусловлен необходимостью автономного развёртывания на потребительском оборудовании без обращения к облачным API. Все модели дообучаются методом параметрически-эффективной адаптации с единым набором гиперпараметров, что обеспечивает честность сравнения.

2.2 Разведочный анализ датасета EXPECT

Для проведения экспериментов по генерации персонализированных упражнений необходим датасет, содержащий аннотированные примеры грамматических ошибок с указанием типа ошибки, уровня владения языком и контекста коррекции. Этим требованиям соответствует датасет EXPECT, опубликованный в открытом репозитории [13, 14]. Датасет предназначен для задачи исправления грамматических ошибок с объяснимой коррекцией и содержит аннотированные пары «ошибочное предложение – исправленное предложение» в формате, пригодном для дообучения языковых моделей.

Общий объём датасета составляет 20016 примеров, распределённых по трём выборкам: обучающая – 15187 примеров, валидационная – 2413 примеров и тестовая – 2416 примеров. Каждый пример включает следующие поля: токенизированное ошибочное предложение (source), токенизированное исправленное предложение (target), тип грамматической ошибки (error_type), индексы позиций коррекции (correction_index), индексы контекстных

токенов-свидетельств (evidence_index), порядок разбора коррекции (predicted_parsing_order) и уровень владения языком автора (origin). Пропуски в данных отсутствуют.

В датасете выделено 15 категорий грамматических ошибок. Распределение типов ошибок представлено на рисунке 1. Наиболее частотными являются Preposition (13,9 %), Collocation (13,4 %) и Verb Tense (12,2 %), которые суммарно составляют около 40 % всех примеров. Наименее представлены Participle (1,1 %), Transitive Verb (1,5 %) и Auxiliary Verb (1,9 %). Выявленный дисбаланс классов отражает реальную частотность ошибок у изучающих английский язык и должен учитываться при обучении моделей посредством стратифицированной выборки или взвешивания функции потерь.

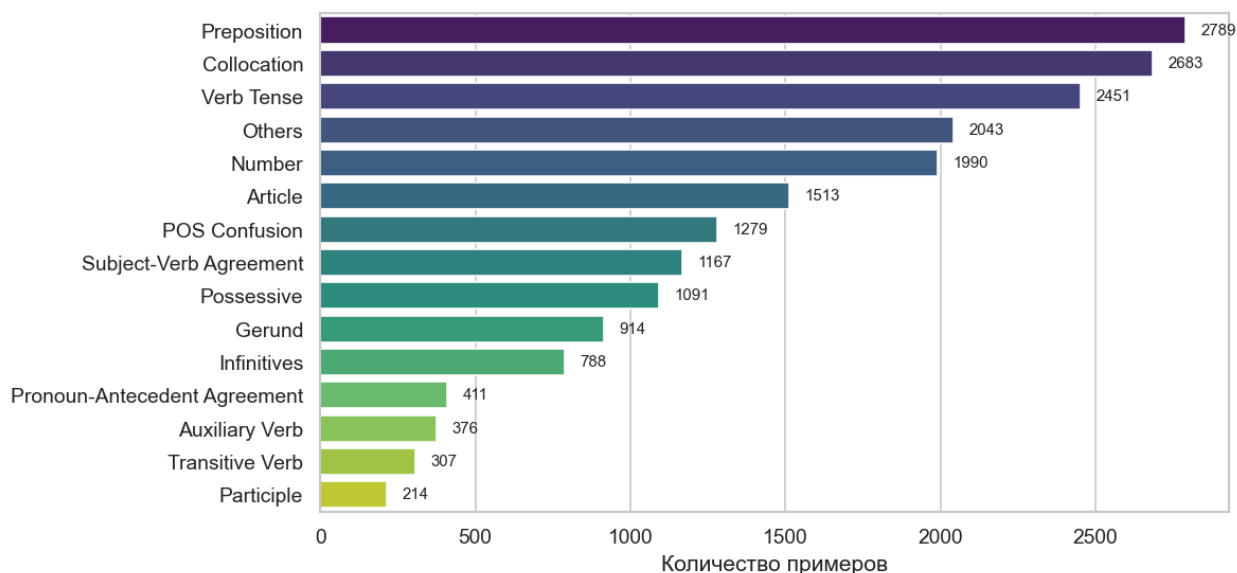


Рисунок 1 – Распределение типов грамматических ошибок

Каждому примеру присвоена метка уровня владения языком (origin). Распределение примеров по уровням приведено на рисунке 2. Выделяются четыре уровня: А (8888 примеров, 44,4 %), В (7678, 38,4 %), С (3076, 15,4 %) и N (374, 1,9 %). Уровень N присутствует исключительно в валидационной и тестовой выборках и характеризуется аномально высокой долей ошибок типа Number (около 29 %), что указывает на специфичность данной подгруппы.

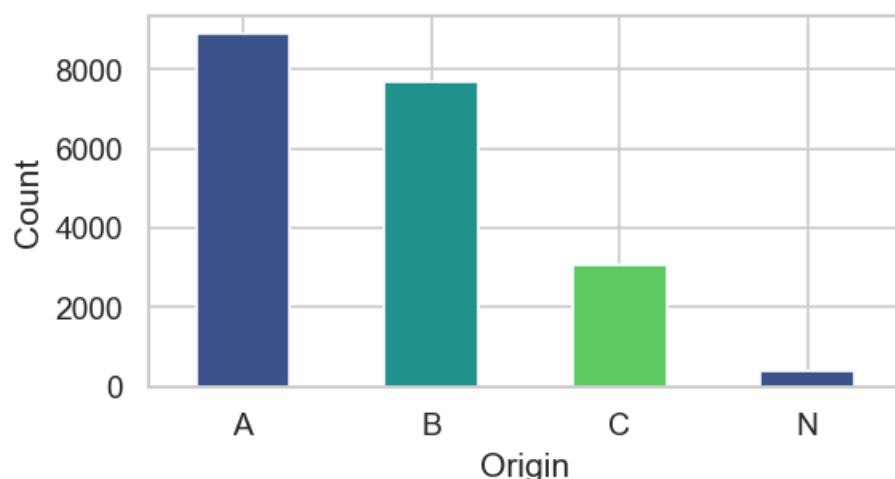


Рисунок 2 – Количество примеров по уровню владения языком

Анализ профилей ошибок в разрезе уровней владения выявляет наличие различий (рисунок 3). На уровне C наблюдается повышенная доля ошибок типа Preposition и Collocation, тогда как на уровне A распределение ошибок более равномерное. Данная закономерность может быть использована для адаптации генерируемых упражнений к уровню ученика.

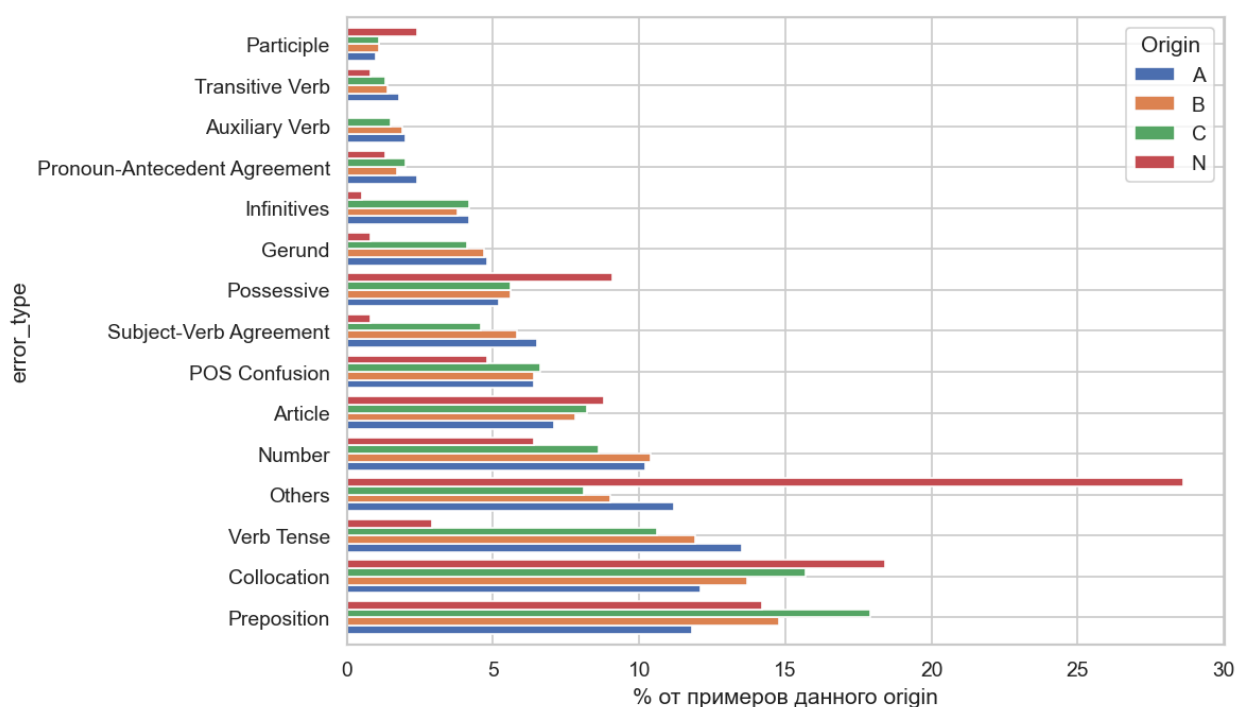


Рисунок 3 – Профиль ошибок по уровню владения языком

Средняя длина предложений в датасете составляет 29 токенов (медиана – 23). Распределение длин правосторонне скошено с длинным хвостом: максимальная длина достигает 236 токенов (рисунок 4, слева). Разница длин между исправленным и ошибочным предложениями (target – source) сконцентрирована в диапазоне от -1 до +1 токена (рисунок 4, справа), что свидетельствует о точечном характере коррекций: в подавляющем

большинстве случаев исправление затрагивает замену, вставку или удаление одного токена, а не масштабное перестроение предложения.

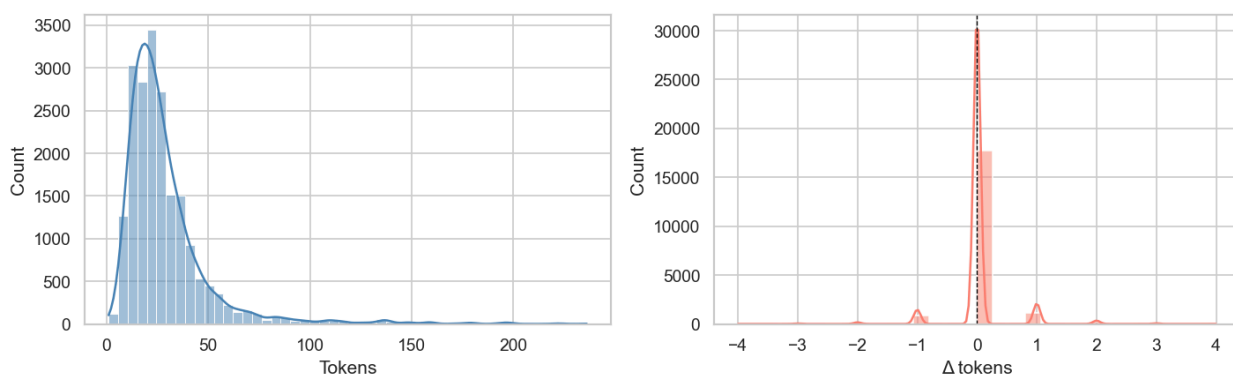


Рисунок 4 – Распределение длины предложений (слева) и разницы длин target – source (справа)

Подавляющее большинство примеров (около 87 %) содержит ровно два correction-индекса, что соответствует формату «позиция ошибочного токена в source и позиция исправленного токена в target». Распределение числа evidence-индексов более широкое – от 0 до 20 (рисунок 5). При этом 29,4 % примеров не содержат evidence-индексов, то есть для них отсутствует явное контекстное обоснование коррекции.

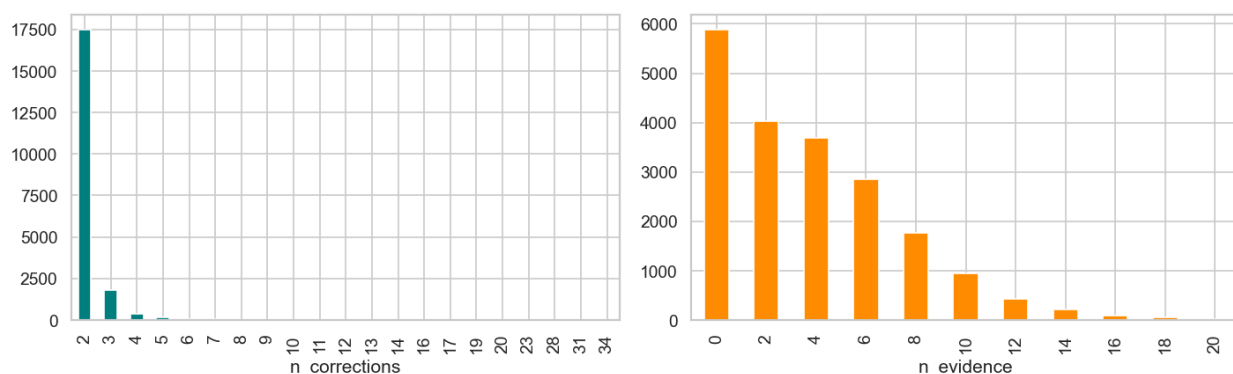


Рисунок 5 – Распределение числа correction-индексов (слева) и evidence-индексов (справа) на пример

Для проверки корректности разбиения на сплиты проводится сравнение распределений типов ошибок по выборкам (рисунок 6). Распределения в целом согласованы, за исключением категории Collocation, доля которой в валидационной выборке (около 22 %) заметно превышает аналогичный показатель в обучающей (около 11 %) и тестовой выборках. Данное расхождение следует учитывать при интерпретации метрик, вычисленных на валидационном множестве.

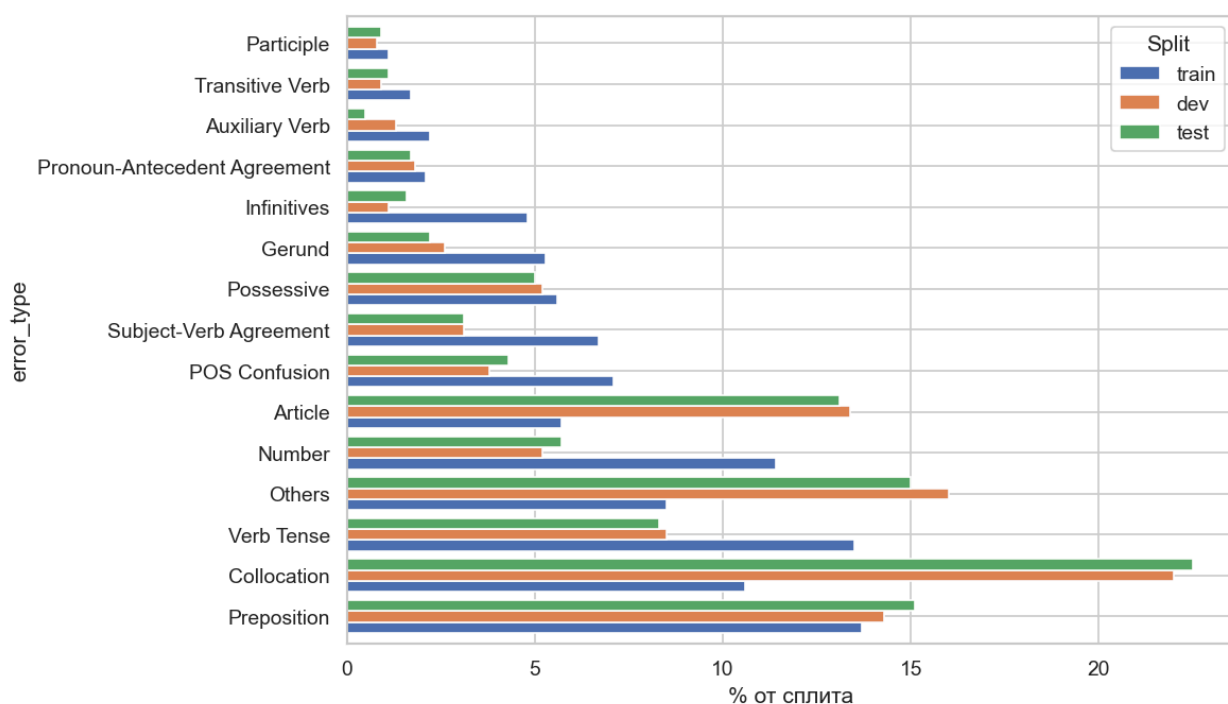


Рисунок 6 – Распределение типов ошибок по выборкам

Дополнительно проводится анализ специального токена [NONE], используемого в датасете для обозначения вставок и удалений. Токен [NONE] в поле source (указывающий на необходимость вставки) встречается в 16,5 % примеров, а в поле target (указывающий на необходимость удаления) – в 7,8 %. Таким образом, вставки пропущенных слов примерно вдвое чаще, чем удаления лишних, что типично для ошибок, изучающих язык, в частности при пропуске предлогов и артиклей.

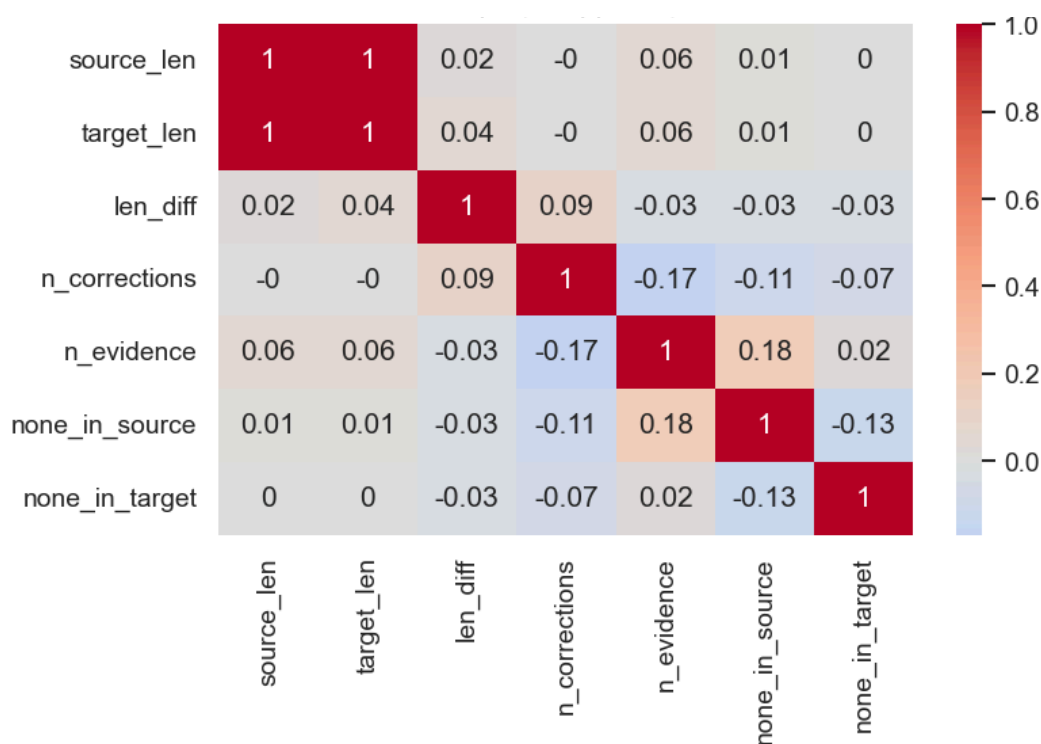


Рисунок 7 – Матрица корреляций числовых признаков

Корреляционный анализ числовых признаков (рисунок 7) показывает отсутствие сильных связей ($|r| < 0,2$), за исключением тривиальной корреляции между длинами source и target ($r \approx 1,0$). Отмечается слабая отрицательная корреляция между числом correction-индексов и числом evidence-индексов ($r = -0,17$): примеры с большим числом коррекций, как правило, имеют меньше контекстных свидетельств.

Помимо основного датасета, репозиторий содержит файлы gector.json (501 пример) и t5.json (500 примеров) с предсказаниями моделей GECToR и T5 соответственно. Структура этих файлов идентична основному датасету (без поля origin), что позволяет использовать их для сравнительного анализа качества автоматической коррекции.

Таким образом, разведочный анализ датасета EXPECT подтверждает его применимость для задачи генерации персонализированных упражнений. Датасет обладает ключевыми свойствами: 15 типизированных категорий ошибок позволяют таргетировать упражнения на конкретные слабые места ученика; уровни владения языком (origin) дают возможность адаптировать сложность; evidence-индексы обеспечивают интерпретируемость коррекций и могут использоваться для генерации объяснений. Основными ограничениями являются дисбаланс классов ошибок, отсутствие evidence-индексов у почти трети примеров, малый объем уровня N и несбалансированность категории Collocation по выборкам.

2.3 Подготовка данных

Подготовка данных для экспериментов включает три последовательных этапа: синтетическую разметку обучающих примеров на основе датасета EXPECT, предобработку полученного файла с проверкой целостности и анализом распределений, а также стратифицированное разбиение на обучающую и валидационную выборки.

Синтетическая разметка. Исходный датасет EXPECT содержит пары «ошибочное предложение – исправленное предложение» с аннотацией типа грамматической ошибки, однако не содержит готовых упражнений в формате, пригодном для обучения генеративной модели. Для формирования обучающей выборки применяется подход синтетической разметки: каждая запись датасета EXPECT преобразуется в полноценный пример упражнения с помощью модели-учителя.

В качестве модели-учителя используется Qwen3-235B-A22B (FP8), доступная через Yandex Cloud Foundational Models API. Выбор обусловлен тем, что модель обладает достаточной ёмкостью для генерации структурированных упражнений высокого качества и доступна в качестве управляемого сервиса, что упрощает масштабирование процесса разметки.

Конвейер разметки реализован следующим образом. Из датасета EXPEST случайным образом отбираются записи, каждой из которых назначается один из четырёх типов заданий: `grammar_choice` (выбор грамматической формы), `transformation` (трансформация предложения), `vocabulary_fill` (заполнение пропусков в тексте) и `functional_matching` (функциональное соотнесение). Назначение типов происходит циклически, что обеспечивает равномерное распределение – по 750 записей на каждый тип при целевом объёме в 3000 примеров.

Для каждой записи формируется пара промптов: системный промпт задаёт роль генератора упражнений, таксономию допустимых категорий ошибок и требования к выходному формату (строгая JSON-схема), а пользовательский промпт содержит конкретные входные данные – текст с ошибкой, тип ошибки, объяснение исправления и запрошенный тип задания. Полные тексты промптов приведены в Приложении В.

Модель-учитель генерирует JSON-объект упражнения, содержащий категорию целевой ошибки, исправленное предложение, инструкцию к заданию, набор из 8–14 вопросов с вариантами ответов (при наличии) и эталонные ответы.

Результат сохраняется в формате JSONL, где каждая строка представляет собой запись в chat-формате: сообщение с ролью `system` (системный промпт), сообщение с ролью `user` (пользовательский промпт с входными данными) и сообщение с ролью `assistant` (сгенерированный JSON-объект упражнения). Дополнительно сохраняется блок метаданных `_meta` с идентификатором записи, исходным и целевым предложениями, типом ошибки и типом задания. Пример итоговой записи приведён в Приложении Г.

Предобработка датасета. Полученный в результате разметки датасет проходит валидацию: каждая запись проверяется на корректность JSON-формата, наличие обязательных полей (сообщения в chat-формате и блок метаданных), а также на то, что ответ модели-учителя содержит валидный JSON-объект упражнения. Все 3000 записей проходят проверку без ошибок.

Далее из записей удаляется служебный блок метаданных, содержащий исходное и целевое предложения, тип ошибки и тип задания. Эти сведения необходимы на этапе оценки качества, но не должны попадать в обучающий контекст модели – при дообучении используются только три сообщения chat-формата (системный промпт, пользовательский запрос и эталонный ответ). Метаданные валидационной части сохраняются отдельно для использования при оценке.

Разбиение на выборки. Разбиение датасета производится в соотношении 90/10 с применением стратификации по полю `task_type`, что гарантирует пропорциональное представительство каждого из четырёх типов заданий в обеих выборках. Фиксация случайного состояния (`random_state = 42`) обеспечивает воспроизводимость. В результате формируются обучающая выборка из 2700 записей (`train.jsonl`) и валидационная выборка из 300 записей (`val.jsonl`).

2.4 Выбор и обоснование моделей генерации упражнений

Ключевым требованием к системе генерации упражнений является автономность: модель должна работать локально, без обращения к внешним API, на оборудовании с ограниченными вычислительными ресурсами. Это требование определяет верхнюю границу размера модели – не более 4 млрд параметров, что позволяет выполнять инференс на серверном GPU среднего уровня (NVIDIA T4/L4) в режиме `fp16` без квантизации.

Дополнительно модель должна удовлетворять следующим критериям: поддержка chat-формата (`system / user / assistant`), способность генерировать структурированный вывод в формате JSON, открытая лицензия, допускающая коммерческое использование, и наличие `instruction-tuned` версии, минимизирующей объём дообучения. На основании этих критериев для сравнительного эксперимента отбираются три модели: Qwen2.5-3B-Instruct, SmolLM3-3B и Phi-3.5-mini-instruct.

Qwen2.5-3B-Instruct – модель семейства Qwen2.5, разработанная Alibaba Cloud и опубликованная в сентябре 2024 года [15]. Архитектура представляет собой `decoder-only Transformer` с 36 слоями и 3,09 млрд параметров. Модель использует `Grouped-Query Attention (GQA)` с 16 головами запросов и 2 головами ключей-значений, что сокращает размер KV-кэша при инференсе. В качестве функции активации применяется `SwiGLU`, позиционное кодирование реализовано через `RoPE (Rotary Position Embedding)`, нормализация – через `RMSNorm`. Контекстное окно составляет 32 768 токенов с возможностью расширения до 128K. Модель предобучена на

18 трлн токенов и прошла этап instruction tuning с акцентом на следование инструкциям и генерацию структурированных выходов, в том числе JSON. По данным разработчиков, Qwen2.5-3B-Instruct превосходит Phi-3.5-mini-instruct на задачах математического рассуждения и кодогенерации при меньшем числе параметров. Лицензия – Qwen Research License, допускающая исследовательское и коммерческое использование.

SmolLM3-3B – модель, разработанная Hugging Face и опубликованная в июле 2025 года [16]. Архитектура построена на базе Llama и включает 3 млрд параметров. Ключевой особенностью является применение техники NoPE (No Position Embedding): в каждом четвёртом слое позиционное кодирование RoPE намеренно отключается, что, по данным авторов, улучшает работу модели на длинных контекстах без потери качества на коротких. Модель использует GQA с 4 группами ключей-значений и поддерживает контекстное окно до 64K токенов (с экстраполяцией через YARN – до 128K). Предобучение выполнено на 11,2 трлн токенов с многоэтапным подходом: базовое обучение на веб-текстах, коде и математике, затем среднее обучение (midtraining) на 140 млрд токенов рассуждений и финальная настройка через SFT и Anchored Preference Optimization (APO). SmolLM3 позиционируется как полностью открытая модель: помимо весов, опубликованы точный состав обучающих данных, конфигурации обучения и промежуточные чекпоинты. Лицензия – Apache 2.0.

Phi-3.5-mini-instruct – модель семейства Phi-3.5, разработанная Microsoft и опубликованная в августе 2024 года [17]. Это dense decoder-only Transformer с 3,8 млрд параметров и контекстным окном 128K токенов. Модель предобучена на 3,4 трлн токенов (синтетические данные и отфильтрованные веб-источники) и прошла трёхэтапное постобучение: supervised fine-tuning (SFT), Proximal Policy Optimization (PPO) и Direct Preference Optimization (DPO). Phi-3.5-mini демонстрирует конкурентоспособные результаты на бенчмарках длинного контекста (RULER: 84,1 в среднем), а также на задачах рассуждения и кодогенерации, на ряде из них превосходя модели с 7–12 млрд параметров. Модель включена в эксперимент как представитель альтернативной архитектурной традиции с отличающимся токенизатором (словарь 32 064 токена) и другой структурой линейных слоёв (fused qkv_proj и gate_up_proj), что позволяет оценить влияние архитектурных различий на качество дообучения методом LoRA. Лицензия – MIT.

Сводная характеристика отобранных моделей приведена в таблице 2.1.

Таблица 2.1 – Сравнительная характеристика отобранных моделей

Характеристика	Qwen2.5-3B-Instruct	SmolLM3-3B	Phi-3.5-mini-instruct
Разработчик	Alibaba Cloud	Hugging Face	Microsoft
Число параметров	3,09 млрд	3 млрд	3,8 млрд
Архитектура	Decoder-only, GQA	Decoder-only, GQA + NoPE	Decoder-only, dense
Контекстное окно	32K (до 128K)	64K (до 128K)	128K
Предобучение	18 трлн токенов	11,2 трлн токенов	3,4 трлн токенов
Лицензия	Qwen Research	Apache 2.0	MIT
Дата публикации	Сентябрь 2024	Июль 2025	Август 2024

Выбор именно трёх моделей обусловлен стремлением обеспечить разнообразие архитектурных решений в рамках одного размерного класса. Qwen2.5-3B и SmolLM3-3B близки по числу параметров, но различаются подходом к позиционному кодированию (RoPE vs NoPE) и объёмом предобучения. Phi-3.5-mini, будучи несколько крупнее (3,8 млрд), представляет другую архитектурную традицию с fused-слоями и иным токенизатором, что позволяет исследовать чувствительность метода LoRA к внутренней структуре модели. Все три модели поддерживают chat-формат, допускают генерацию JSON и удовлетворяют требованиям по лицензированию.

2.5 Дообучение моделей

Дообучение трёх отобранных моделей выполняется методом LoRA на обучающей выборке из 2700 примеров в chat-формате.

Все модели загружаются в формате fp16 (16-битная точность) без применения квантизации. Решение отказаться от квантизации обусловлено несовместимостью версий CUDA в используемой вычислительной среде. Модели в fp16 занимают около 6 ГБ видеопамати, а с LoRA-адаптерами и включённым gradient checkpointing пиковое потребление составляет 12-13 ГБ, что укладывается в доступные ресурсы.

Целевые слои для адаптации определяются автоматически: из архитектуры модели извлекаются все линейные слои за исключением выходной головы (lm_head). В Qwen2.5-3B и SmolLM3-3B обнаруживается 7 уникальных типов целевых модулей (отдельные слои для query, key, value, output проекций внимания и слоёв feed-forward), тогда как в Phi-3.5-mini — только 4, поскольку её архитектура использует объединённые (fused) проекции qkv_proj и gate_up_proj. Это различие непосредственно влияет на

гранулярность адаптации и, как следствие, на число обучаемых параметров: у Qwen2.5-3B и SmolLM3-3B доля обучаемых параметров составляет 3,74-3,78 % от общего числа, тогда как у Phi-3.5-mini – 2,57 %.

Гиперпараметры обучения унифицированы для всех трёх моделей, за исключением максимальной длины последовательности, что обеспечивает корректность сравнения: различия в результатах обусловлены свойствами моделей, а не настройками обучения. Ранг адаптации устанавливается равным $r = 64$ как баланс между ёмкостью и вычислительными затратами, коэффициент масштабирования – $\alpha = 128$ (стандартное соотношение $\alpha = 2r$), dropout на адаптерах – 0,05. Обучение проводится в течение 3 эпох со скоростью обучения 2×10^{-4} , cosine-планировщиком и линейным прогревом на первых 5 % шагов. Регуляризация весов составляет 0,01. Размер батча на устройство равен 2 с накоплением градиентов за 8 шагов, что даёт эффективный размер батча 16. Оценка на валидационной выборке выполняется каждые 50 шагов.

Максимальная длина последовательности для Qwen2.5-3B и SmolLM3-3B устанавливается равной 2048 токенов, что с запасом покрывает среднюю длину примера (1978 и 1987 токенов соответственно). Для Phi-3.5-mini длина увеличена до 2560 токенов, поскольку токенизатор этой модели менее компактен и генерирует в среднем 2329 токенов на пример.

Обучение реализовано с использованием библиотек Transformers, PEFT и TRL. Входные данные преобразуются в текстовый формат, что обеспечивает корректную разметку ролей (system, user, assistant) для каждой модели. Воспроизводимость гарантируется фиксацией случайного состояния для всех генераторов случайных чисел.

Сводная конфигурация дообученных моделей приведена в таблице 2.2.

Таблица 2.2 – Конфигурация дообученных моделей

Параметр	Qwen2.5-3B	SmolLM3-3B	Phi-3.5-mini
Обучаемые (LoRA)	119,7 млн (3,74 %)	120,9 млн (3,78 %)	100,7 млн (2,57 %)
LoRA rank / alpha	64 / 128	64 / 128	64 / 128
Max seq length	2048	2048	2560
Средняя длина примера	1978 токенов	1987 токенов	2329 токенов
Число целевых модулей	7	7	4
Эпохи / LR	3 / 2×10^{-4}	3 / 2×10^{-4}	3 / 2×10^{-4}
Эффективный батч	16	16	16

Доля обучаемых параметров составляет менее 4 % от общего числа параметров модели, что подтверждает эффективность метода LoRA. Phi-3.5-mini имеет меньшую долю обучаемых параметров (2,57 %) несмотря на больший общий размер модели: это следствие объединённых линейных слоёв, в которых LoRA-адаптер покрывает крупный модуль целиком, но не может адаптировать его подкомпоненты по отдельности.

Кривые обучения для трёх моделей представлены на рисунке 8. Qwen2.5-3B и SmolLM3-3B демонстрируют быструю сходимость: train loss опускается ниже 0,15 к 100-му шагу, eval loss стабилизируется на уровне 0,2. Phi-3.5-mini стартует со значительно более высокого уровня потерь (~11) и к завершению обучения снижается лишь до ~1,7, не достигая уровня двух других моделей.

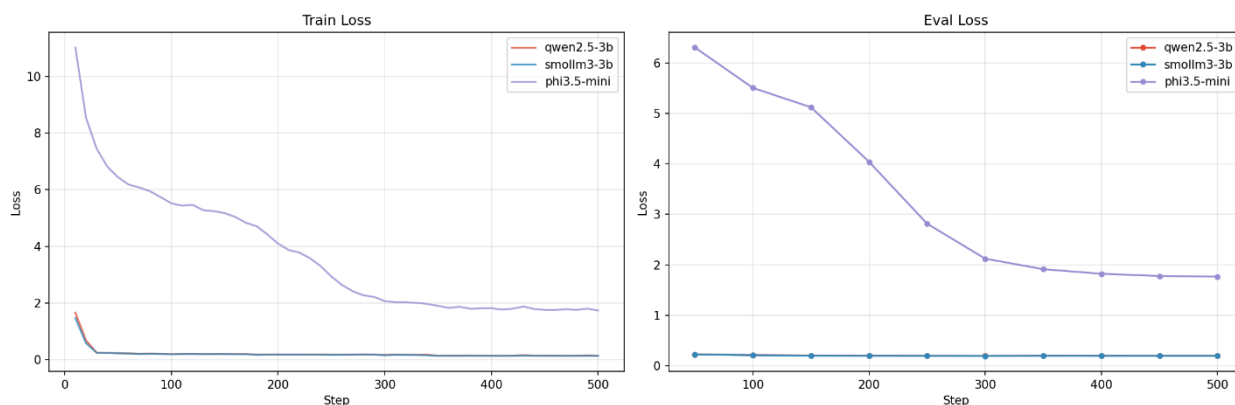


Рисунок 8 – Кривые обучения (Train Loss и Eval Loss)

Финальные значения функции потерь приведены в таблице 2.3.

Таблица 2.3 – Финальные метрики обучения

Модель	Final Train Loss	Final Eval Loss	Steps
Qwen2.5-3B-Instruct	0,141	0,209	504
SmolLM3-3B	~0,08	~0,10	504
Phi-3.5-mini-instruct	~1,7	~1,8	507

Разрыв в качестве обучения Phi-3.5-mini обусловлен совокупностью факторов. Во-первых, менее компактный токенизатор генерирует в среднем 2329 токенов на пример (против 1978 у Qwen), что при ограничении длины последовательности приводит к усечению части обучающих примеров. Во-вторых, архитектура с объединёнными проекциями (fused qkv_proj и gate_up_proj) предоставляет LoRA лишь 4 точки адаптации вместо 7, что снижает гранулярность настройки. В-третьих, существенно более высокий стартовый loss свидетельствует о том, что базовая модель хуже подготовлена

к данному формату задач, и трёх эпох оказывается недостаточно для полного усвоения структуры длинных JSON-упражнений.

SmolLM3-3B достигает наименьших значений loss (train $\sim 0,08$, eval $\sim 0,10$), что может указывать на начальные признаки переобучения. Тем не менее, как показывает последующая оценка качества, модель сохраняет высокое качество генерации на валидационной выборке.

По итогам обучения для каждой модели сохраняются LoRA-адаптеры – компактные файлы весов, не превышающие 500 МБ, – которые при инференсе загружаются совместно с исходными замороженными весами базовой модели.

2.6 Экспериментальная оценка качества методов

Экспериментальная оценка выполняется на валидационной выборке из 300 примеров (10 % от размеченного датасета), сформированной со стратификацией по типу задания. На этой же выборке сравниваются пять методов: три дообученные LLM (Qwen2.5-3B, SmolLM3-3B, Phi-3.5-mini) и два шаблонных бейзлайна (Fill-in-the-Blanks и Sentence Reconstruction), описанные в разделе 2.1. Набор метрик – BERTScore F1, Perplexity и композитная метрика Task-Relevance (TR) с пятью подметриками – также определён в разделе 2.1.

Для LLM-моделей инференс выполняется с адаптерами LoRA, полученными на этапе дообучения, и параметрами генерации $temperature = 0,3$, $max_new_tokens = 2048$. Для каждого примера валидационной выборки модель получает тот же системный и пользовательский промпт, что и на этапе разметки (Приложение В), и генерирует JSON-объект упражнения. Для шаблонных бейзлайнов в качестве входа используется поле `target` соответствующей записи EXPECT.

BERTScore F1 вычисляется сравнением сгенерированного JSON-упражнения с эталонным ответом из валидационной выборки с помощью модели `distilbert-base-uncased` чанковым методом (размер чанка – 3000 символов). Perplexity рассчитывается как экспонента от среднего значения `cross-entropy loss` на валидационной выборке; для шаблонных бейзлайнов метрика неприменима, поскольку их генерация детерминирована. Task-Relevance оценивается путём парсинга выходного JSON и проверки пяти бинарных условий для каждого примера с последующим усреднением.

Сводные результаты по всем пяти методам представлены в таблице 2.4.

Таблица 2.4 – Результаты оценки методов генерации на валидационной выборке (n = 300)

Метод	BERT F1	PPL	JSON Valid	Task Match	Items OK	Answers OK	Err. Cat	TR
Qwen2.5-3B	0,9085	1,22	98,7 %	85,3 %	91,3 %	95,0 %	98,7 %	93,8 %
SmolLM3-3B	0,9089	1,23	96,0 %	83,7 %	89,3 %	94,0 %	96,0 %	91,8 %
Phi-3.5-mini	0,8806	4,70	80,7 %	80,0 %	2,0 %	80,0 %	80,7 %	64,7 %
Fill-in-the-Blanks	—	—	100 %	0 %	95,0 %	95,0 %	95,0 %	77,0 %
Sentence Reconstruction	—	—	100 %	0 %	39,0 %	39,0 %	39,0 %	43,4 %

Детализация подметрик Task-Relevance по всем пяти методам визуализирована на рисунке 9, рейтинг методов по композитной метрике – на рисунке 10.

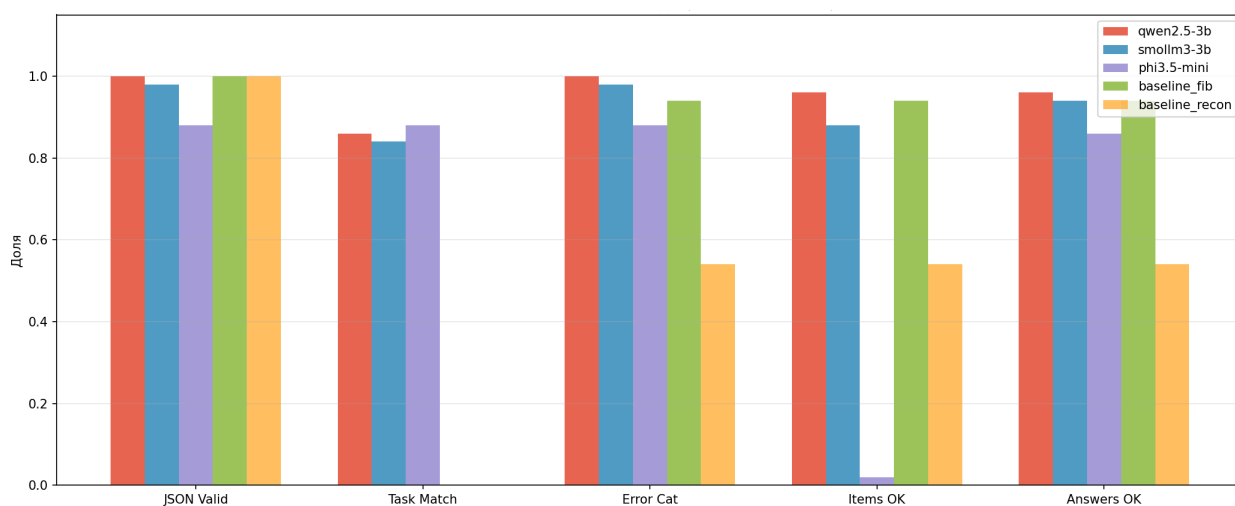


Рисунок 9 – Детализация подметрик Task-Relevance по методам

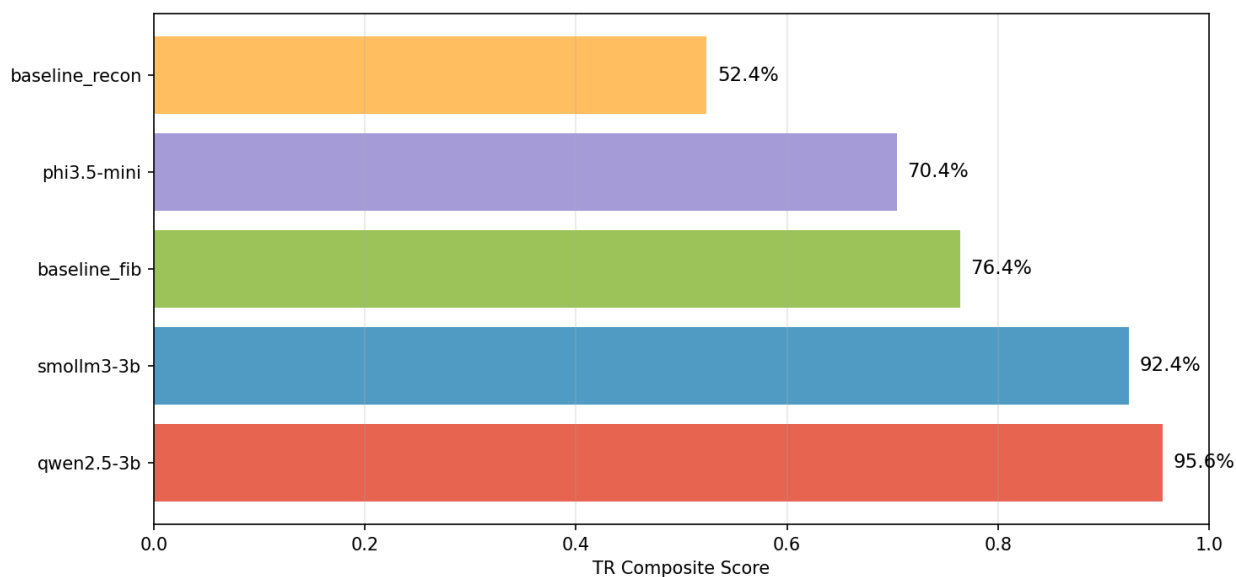


Рисунок 10 – Рейтинг методов по Task-Relevance Composite

Представленные результаты позволяют охарактеризовать поведение каждого метода в отдельности и сопоставить два класса подходов между собой. Далее приводится последовательный разбор: сначала рассматриваются три дообученные LLM, затем – шаблонные бейзлайны, после чего формулируются обобщающие наблюдения.

Qwen2.5-3B демонстрирует лучший агрегированный результат: TR = 93,8 %, BERTScore F1 = 0,9085, Perplexity = 1,22. Модель генерирует валидный JSON в 98,7 % случаев, укладывается в допустимый диапазон количества items в 91,3 % и корректно воспроизводит формат всех четырёх типов заданий. Медианное количество items на упражнение – 12, что соответствует верхней границе требуемого диапазона и обеспечивает достаточную плотность учебного материала.

SmolLM3-3B показывает практически идентичное семантическое качество (BERTScore F1 = 0,9089 против 0,9085 у Qwen) при сопоставимой Perplexity (1,23 против 1,22). Разница в композитной метрике (91,8 % против 93,8 %) обусловлена преимущественно потерями в структурной корректности: JSON Valid снижен до 96,0 %, Items OK – до 89,3 %. В части случаев модель генерирует 5-7 items вместо минимальных 8, что указывает на менее строгое следование требованиям к объёму.

Phi-3.5-mini демонстрирует заметное снижение по композитной метрике TR = 64,7%, при этом критически низкое значение принимает подметрика Items OK – 2,0%. Несмотря на увеличение максимальной длины последовательности до 2560 токенов, модель генерирует лишь 1-5 items вместо требуемых 8-14. При этом Task Match = 80,0% показывает, что тип задания определяется корректно, а основной сбой связан именно с удержанием длинной структуры JSON-упражнения при генерации. Повышенный Perplexity = 4,70 дополнительно указывает на более низкую уверенность модели в данном формате данных.

Шаблонный бейзлайн Fill-in-the-Blanks достигает TR = 77,0 %. Высокие значения по четырём из пяти подметрик объясняются детерминированностью генерации: JSON всегда валиден, количество items и наличие ответов гарантированы конструкцией. Нулевое значение Task Match отражает принципиальное ограничение подхода – бейзлайн всегда выдаёт единственный тип задания (fill-in-the-blanks) независимо от запрошенного в промпте. Coverage составляет 95 % (285 из 300 примеров пригодны для генерации), для остальных 5 % коррекция выходит за рамки возможностей шаблона.

Sentence Reconstruction показывает наименьший результат (TR = 43,4 %). Основной причиной является низкий coverage – 39 % (117 из 300 примеров), – поскольку метод фильтрует предложения длиной 4-20 токенов, а большинство записей в EXPECT содержит более длинные предложения. Это делает подход неприменимым для значительной части входных данных.

В совокупности эти наблюдения показывают, что дообученные LLM Qwen2.5-3B и SmolLM3-3B превосходят шаблонные бейзлайны по трём ключевым параметрам: объёму контента на упражнение (10-12 items против 1), разнообразию форматов (4 типа против 1) и способности адаптироваться под запрошенный тип задания и тип ошибки. Шаблонные методы выигрывают только в детерминированности – их вывод структурно предсказуем и не требует постобработки, что делает их пригодными в качестве резервного механизма на случай сбоев LLM. Результаты эксперимента позволяют сделать предварительный выбор оптимальной модели для системы генерации.

2.7 Обоснование эффективности предложенного решения

Результаты экспериментальной оценки (раздел 2.6) демонстрируют существенное превосходство дообученных LLM-моделей над шаблонными бейзлайнами, однако для обоснованного выбора оптимальной модели необходимо оценить статистическую значимость различий, систематизировать преимущества и ограничения каждого подхода и сформулировать критерии итогового решения.

Центральным количественным показателем сравнения является композитная метрика Task-Relevance (TR), агрегирующая пять подметрик в единый индикатор качества генерации. Разница между лучшей LLM (Qwen2.5-3B, TR = 93,8 %) и лучшим бейзлайном (Fill-in-the-Blanks, TR = 77,0 %) составляет 16,8 процентных пунктов. При этом превосходство LLM-моделей носит не только количественный, но и качественный характер: дообученные модели генерируют упражнения четырёх различных типов (grammar_choice, transformation, vocabulary_fill, functional_matching) с 10-12 items на упражнение, тогда как Fill-in-the-Blanks ограничен единственным типом задания с одним item. Sentence Reconstruction (TR = 43,4 %) дополнительно ограничен coverage в 39 % – более половины входных данных отсеивается на этапе фильтрации по длине предложения, что делает метод непригодным для использования в продуктивной системе.

Различие между Qwen2.5-3B (TR = 93,8 %) и SmolLM3-3B (TR = 91,8 %) составляет 2,0 процентных пункта. По семантическим метрикам модели

практически неразличимы: BERTScore F1 (0,9085 против 0,9089) и Perplexity (1,22 против 1,23), что свидетельствует о сопоставимом качестве генерируемого текста. Наблюдаемая разница в TR обусловлена преимущественно структурными подметриками: JSON Valid (98,7 % против 96,0 %) и Items OK (91,3 % против 89,3 %), где Qwen2.5-3B стабильнее удерживает заданную структуру упражнения.

Разрыв между двумя лидерами и Phi-3.5-mini (TR = 64,7%) замечен по всем подметрикам. Особенно резко проседает Items OK = 2,0%, что объясняется сочетанием менее компактного токенизатора, меньшего числа целевых модулей для LoRA-адаптации и более высокого стартового loss, из-за которого модель за три эпохи не успевает надежно усвоить структуру длинных JSON-упражнений. Это подтверждает, что совместимость токенизатора с длиной обучающих данных является критически важным фактором при выборе модели для задач генерации структурированного вывода.

Динамика обучения дополняет картину: Qwen2.5-3B и SmolLM3-3B демонстрируют быструю сходимость с финальным train loss 0,141 и ~0,08 соответственно, тогда как Phi-3.5-mini снижается лишь до ~1,7. Низкий train loss SmolLM3-3B при более высоких потерях на структурных метриках указывает на то, что модель хорошо выучивает семантику эталонных ответов, но менее стабильна в воспроизведении точной структуры JSON. Qwen2.5-3B занимает промежуточное положение по train loss, однако обеспечивает лучший баланс между семантическим качеством и структурной корректностью.

На основании совокупности результатов Qwen2.5-3B-Instruct выбирается в качестве оптимальной модели для системы генерации упражнений. Выбор обусловлен наивысшим TR Composite (93,8 %), надёжной генерацией валидного JSON (98,7 %), корректным воспроизведением всех четырёх типов заданий, низкой Perplexity (1,22), свидетельствующей об уверенности модели, а также компактным размером LoRA-адаптера, не превышающим 500 МБ. SmolLM3-3B рассматривается как резервная альтернатива благодаря идентичному семантическому качеству и полностью открытой лицензии (Apache 2.0), но уступает Qwen по стабильности структурного вывода. Phi-3.5-mini не рекомендуется для данной задачи в текущей конфигурации обучения.

Шаблонные методы, несмотря на более низкие значения TR, сохраняют практическую ценность как резервный механизм: в случаях, когда

LLM генерирует невалидный JSON или не укладывается в ограничения по времени ответа, система может переключиться на детерминированную генерацию Fill-in-the-Blanks, обеспечивающую гарантированную корректность выхода.

Стоит отметить элементы новизны проведённого исследования. В рамках работы выполнено сравнение нейросетевых и шаблонных методов генерации упражнений на едином датасете с единым набором метрик, что позволяет объективно оценить вклад каждого подхода. Кроме того, реализован конвейер, в котором тип и содержание задания определяются конкретной грамматической ошибкой ученика, а не задаются вручную. Наконец, продемонстрировано, что компактные модели (до 4 млрд параметров), дообученные на сравнительно небольшом датасете (3000 примеров), достигают уровня качества, достаточного для практического использования ($TR > 90 \%$, $BERTScore > 0,90$), при полной автономности – без обращения к облачным API.

Таким образом, эксперимент подтверждает основную гипотезу работы и обосновывает выбор Qwen2.5-3B-Instruct + LoRA в качестве ядра системы генерации персонализированных упражнений.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Kasneci E. et al. ChatGPT for good? On opportunities and challenges of large language models for education // Learning and Individual Differences. — 2023. — Vol. 103. — P. 102274.
2. Al-Khatib M. A. Artificial intelligence in language instruction: impact on English learning achievement, L2 motivation, and self-regulated learning // Frontiers in Psychology. — 2023. — Vol. 14. — P. 1261955.
3. Cui P., Sachan M. Adaptive and Personalized Exercise Generation for Online Language Learning [Электронный ресурс] // arXiv preprint. — 2023. — URL: <https://arxiv.org/abs/2306.02457> (дата обращения: 15.04.2026).
4. Chen X., Zou D., Xie H. et al. Two decades of artificial intelligence in education: Contributors, collaborations, research topics, challenges, and future directions // Educational Technology & Society. — 2022. — Vol. 25, No. 1. — P. 28–47.
5. Grand View Research. Online Language Learning Market Size | Industry Report 2030 [Электронный ресурс]. — URL: <https://www.grandviewresearch.com/industry-analysis/online-language-learning-market-report> (дата обращения: 01.04.2026).
6. Duolingo Blog. At Duolingo, humans and AI work together to create a high-quality learning experience [Электронный ресурс]. — 2024. — URL: <https://blog.duolingo.com/how-duolingo-experts-work-with-ai/> (дата обращения: 01.04.2026).
7. Duolingo Blog. How Duolingo uses AI to create lessons faster [Электронный ресурс]. — 2025. — URL: <https://blog.duolingo.com/large-language-model-duolingo-lessons/> (дата обращения: 01.04.2026).
8. ELSA Speak. Product Overview [Электронный ресурс]. — URL: <https://elsaspeak.com/en/product/> (дата обращения: 01.04.2026).
9. European School Education Platform. Survey on language learning — Results [Электронный ресурс]. — URL: <https://school-education.ec.europa.eu/en/discover/surveys/survey-language-learning-results> (дата обращения: 01.04.2026).
10. Susanti Y. et al. Automatic Distractor Generation for Multiple-Choice English Vocabulary Questions // Research and Practice in Technology Enhanced Learning. — 2018. — Vol. 13, No. 15.
11. Zanetti A., Volodina E., Graën J. Automatic Generation of Exercises for Second Language Learning from Parallel Corpus Data // International Journal of TESOL Studies. — 2021. — Vol. 3, No. 2. — P. 55–71.

12. Hu E. J. et al. LoRA: Low-Rank Adaptation of Large Language Models [Электронный ресурс] // arXiv preprint. — 2021. — URL: <https://arxiv.org/abs/2106.09685> (дата обращения: 15.04.2026).
13. Fei Y., Cui L., Yang S., Lam W., Lan Z., Shi S. Enhancing Grammatical Error Correction Systems with Explanations // Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL). — 2023. — P. 7176–7187.
14. Fei Y. Explainable GEC: Dataset and Code [Электронный ресурс]. — URL: https://github.com/lorafei/Explainable_GEC (дата обращения: 15.04.2026).
15. Yang A. et al. Qwen2.5 Technical Report [Электронный ресурс] // arXiv preprint. — 2024. — URL: <https://arxiv.org/abs/2412.15115> (дата обращения: 15.04.2026).
16. Bakouch E. et al. SmolLM3: smol, multilingual, long-context reasoner [Электронный ресурс] // Hugging Face Blog. — 2025. — URL: <https://huggingface.co/blog/smollm3> (дата обращения: 15.04.2026).
17. Abdin M. et al. Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone [Электронный ресурс] // arXiv preprint. — 2024. — URL: <https://arxiv.org/abs/2404.14219> (дата обращения: 15.04.2026).

Приложение А

Листинг шаблонного метода Fill-in-the-Blanks

```
def extract_blank_word(row: pd.Series) -> Dict:
    """
    Извлекает целевое слово для пропуска из записи датасета.
    Определяет тип коррекции (replacement / insertion / deletion)
    и позицию пропуска в целевом предложении.
    """
    target_tokens = row['target'] if isinstance(row['target'], list) \
        else row['target'].split(',')
    target_clean = [t for t in target_tokens if t != '[NONE]']
    correction_idx = row['correction_index']
    correction_type = row.get('correction_type', 'replacement')

    result = {
        'suitable': True,
        'correction_type': correction_type,
        'blank_word': None,
        'wrong_answer': None,
        'blank_position': None,
    }

    if correction_type == 'replacement':
        if len(correction_idx) > 0:
            pos = correction_idx[0]
            if pos < len(target_clean):
                result['blank_word'] = target_clean[pos]
                result['blank_position'] = pos
            if len(row['evidence_index']) > 0:
                evidence_pos = row['evidence_index'][0]
                source_clean = [t for t in row['source']
                               if t != '[NONE]']
                if evidence_pos < len(source_clean):
                    result['wrong_answer'] = source_clean[evidence_pos]

    elif correction_type == 'insertion':
        if len(correction_idx) > 0:
            pos = correction_idx[0]
            if pos < len(target_clean):
                result['blank_word'] = target_clean[pos]
                result['blank_position'] = pos
                result['wrong_answer'] = ""

    elif correction_type == 'deletion':
```



```

    result['suitable'] = False

    return result

def create_blank_template(row: pd.Series) -> Optional[str]:
    """
    Создаёт текст упражнения с пропуском (_____) на месте
    целевого слова.
    """
    if not row.get('is_suitable', False):
        return None

    blank_pos = int(row['blank_position'])
    target_tokens = row['target'] if isinstance(row['target'], list) \
        else row['target'].split(',')
    target_clean = [t for t in target_tokens if t != '[NONE]']

    template_tokens = target_clean.copy()
    template_tokens[blank_pos] = '_____'

    return detokenize(template_tokens)

def generate_spacy_distractors(row, pool, n_distractors=3):
    """
    Генерирует дистракторы на основе POS-категории
    правильного ответа через spaCy.
    Приоритет: слова той же POS, затем — других POS.
    """
    correct_answer = row['blank_word']
    correct_pos = get_pos_spacy(correct_answer)
    distractors = set()

    # Сначала добавляем слова той же POS-категории
    if correct_pos in pool:
        candidates = [w for w in pool[correct_pos]
                       if w != correct_answer]
        random.shuffle(candidates)
        for w in candidates[:n_distractors]:
            distractors.add(w)

    # Дополняем из других POS-категорий при нехватке
    if len(distractors) < n_distractors:
        for pos_cat, words in pool.items():

```

```

    if pos_cat != correct_pos:
        for w in words:
            if w != correct_answer and w not in distractors:
                distractors.add(w)
                if len(distractors) >= n_distractors:
                    break
        if len(distractors) >= n_distractors:
            break

    return list(distractors)[:n_distractors]

def build_exercise_spacy(row: pd.Series, pool: Dict) -> Optional[Dict]:
    """Собирает упражнение с POS-фильтрованными дистракторами."""
    if not row.get('is_suitable', False) or \
        row.get('exercise_template') is None:
        return None

    correct_answer = row['blank_word']
    distractors = generate_spacy_distractors(row, pool,
                                             n_distractors=3)
    options = [correct_answer] + distractors
    random.shuffle(options)

    return {
        'exercise_id': int(row.name),
        'template': row['exercise_template'],
        'correct_answer': correct_answer,
        'options': options,
        'error_type': row['error_type'],
        'correction_type': row['correction_type'],
        'correct_pos': get_pos_spacy(correct_answer),
    }

```

Приложение Б

Листинг шаблонного метода Sentence Reconstruction

```
def get_tokens_from_row(row: pd.Series,
                        column: str = 'target') -> List[str]:
    """
    Извлекает токены из записи датасета.
    Удаляет служебные токены [NONE] и пустые значения.
    """
    tokens_raw = row.get(column, "")

    if isinstance(tokens_raw, str):
        tokens = tokens_raw.split(',')
    elif isinstance(tokens_raw, list):
        tokens = tokens_raw
    else:
        return []

    tokens = [t.strip() for t in tokens
              if t.strip() and t != '[NONE]']
    return tokens


def is_suitable_for_reconstruction(row: pd.Series) -> bool:
    """
    Проверяет пригодность предложения для упражнения
    на реконструкцию. Фильтрует по длине: 4–20 токенов.
    """
    tokens = get_tokens_from_row(row, 'target')
    return 4 <= len(tokens) <= 20


def shuffle_tokens(tokens: List[str],
                  difficulty: str = 'medium') -> List[str]:
    """
    Перемешивает токены с учётом уровня сложности.
    easy — первый и последний токены остаются на месте
    medium — полное случайное перемешивание
    hard — полное случайное перемешивание
    """
    if len(tokens) <= 2:
        return tokens.copy()

    shuffled = tokens.copy()
```

```

if difficulty == 'easy':
    middle = shuffled[1:-1]
    random.shuffle(middle)
    shuffled = [shuffled[0]] + middle + [shuffled[-1]]
else:
    random.shuffle(shuffled)

return shuffled

def create_reconstruction_exercise(
    row: pd.Series,
    difficulty: str = 'medium') -> Optional[Dict]:
    """
    Создаёт упражнение на восстановление порядка слов.
    """
    if not is_suitable_for_reconstruction(row):
        return None

    tokens = get_tokens_from_row(row, 'target')
    shuffled = shuffle_tokens(tokens, difficulty)

    return {
        'exercise_id': int(row.name),
        'type': 'sentence-reconstruction',
        'error_type': row.get('error_type', 'Unknown'),
        'difficulty': difficulty,
        'shuffled_tokens': shuffled,
        'correct_order': tokens,
        'correct_sentence': row.get('target_text', ""),
        'num_tokens': len(tokens),
    }

```

Приложение В

Промпты для синтетической разметки обучающего датасета

Системный промпт (SYSTEM_PROMPT):

You are an expert English exercise generator.

Your task:

Generate exactly ONE exercise based on the user's grammar error and the requested task type.

TAXONOMY:

- Adjective Form
- Noun Inflection
- Noun Number
- Tense Usage
- Verb Form
- Verb Inflection
- Verb Agreement
- Verb Tense
- Part of Speech
- Adverb
- Determiner
- Particle
- Preposition
- Spelling
- Word Order
- No Error

(Use only these labels.)

Output requirements:

- JSON only.
- Strict adherence to the requested "task_type".
- NO bundles. Just a single exercise task.
- Instructions and content in English.

Safety:

- No PII.
- Correct English grammar in examples.

Шаблон пользовательского промпта (USER_PROMPT_TEMPLATE):

Generate exactly one exercise task matching the requested type.

INPUT DATA:

user_id: "{user_id}"

message_content: "{message_content}"

```
grammar_error: "{grammar_error}"
explanation: "{explanation}"
llm_confidence: {llm_confidence}
task_type: "{task_type}"
```

OUTPUT JSON SCHEMA (strict; JSON only):

```
{
  "target_error_category": "string",
  "corrected_sentence": "string",
  "task": {
    "type": "string",
    "instruction_en": "string",
    "content_en": {
      "context_text": "string" | null,
      "items": [
        {
          "question_en": "string",
          "options_en": ["string"] | null,
          "student_answer_en": "string"
        }
      ],
      "word_bank": ["string"] | null
    }
  }
}
```

EXERCISE RULES (Strict Formatting):

1. Vocabulary Fill (Long text):
 - Generate a coherent text with 12-20 gaps.
 - Provide a "word_bank" list.
 - word_bank must contain UNIQUE words only.
2. Matching / Categories:
 - Provide 8-12 pairs or 10-16 category items.
 - Format: "***_1-D_***" or "***_Furniture: Sofa_***".
3. Grammar Choice (10-14 items):
 - MINIMUM 10 items, MAXIMUM 14 items.
 - Provide exactly 2-3 options per item.
 - student_answer_en: full sentence with bold answer AND options.
4. Transformations (8-12 items):
 - MINIMUM 8 items.
 - student_answer_en: full rewritten sentence with

bolded changes.

5. Reading (True/False):

- Provide text (120-180 words) in context_text.
- 8 statements in items.

6. Functional Matching:

- 8-10 items with 4 labelled choices each.
- Format: "***_1-C_***".

7. Writing Sample:

- Provide context_text (prompt).
- Student answer: 60-90 words.

GENERAL RULES:

- Target the specific grammar_error primarily, but expand context to the general topic of the user's message.
- If llm_confidence < 0.75, generate a "Verification/Choice" task first.
- Return ONLY valid JSON.

Приложение Г

Пример записи синтетически размеченного датасета

```
{
  "target_error_category": "Gerund",
  "corrected_sentence": "One thousand people of eighteen years and over were
questioned in my town about their ways of taking.",
  "task": {
    "type": "grammar_choice",
    "instruction_en": "Choose the correct form of the word to complete each
sentence. Focus on the correct use of gerunds after prepositions and expressions.",
    "content_en": {
      "context_text": null,
      "items": [
        {
          "question_en": "She was praised for her dedication to ____ the community.
(serve/serving/served)",
          "options_en": [
            "serve",
            "serving",
            "served"
          ],
          "student_answer_en": "She was praised for her dedication to **_serving_**
(serve/serving/served).",
        },
        {
          "question_en": "They apologized for ____ late to the meeting.
(arrive/arriving/arrived)",
          "options_en": [
            "arrive",
            "arriving",
            "arrived"
          ],
          "student_answer_en": "They apologized for **_arriving_**
(arrive/arriving/arrived).",
        },
        {
          "question_en": "He is interested in ____ a new language.
(learn/learning/learned)",
          "options_en": [
            "learn",
            "learning",
            "learned"
          ],
        }
      ]
    }
  }
}
```



```

    "student_answer_en": "He is interested in **_learning_**
(learn/learning/learned).",
    },
    {
        "question_en": "We discussed the possibility of ____ early.
(leave/leaving/left)",
        "options_en": [
            "leave",
            "leaving",
            "left"
        ],
        "student_answer_en": "We discussed the possibility of **_leaving_**
(leave/leaving/left).",
    },
    {
        "question_en": "She is tired of ____ the same mistakes.
(make/making/made)",
        "options_en": [
            "make",
            "making",
            "made"
        ],
        "student_answer_en": "She is tired of **_making_** (make/making/made).",
    },
    {
        "question_en": "They are excited about ____ to Japan next month.
(go/going/went)",
        "options_en": [
            "go",
            "going",
            "went"
        ],
        "student_answer_en": "They are excited about **_going_**
(go/going/went).",
    },
    {
        "question_en": "He insisted on ____ the bill himself. (pay/paying/paid)",
        "options_en": [
            "pay",
            "paying",
            "paid"
        ],
        "student_answer_en": "He insisted on **_paying_** (pay/paying/paid).",
    },
    {

```

```

    "question_en": "I am looking forward to ____ my grandparents this weekend.
(see/seeing/saw)",
    "options_en": [
        "see",
        "seeing",
        "saw"
    ],
    "student_answer_en": "I am looking forward to **_seeing_**
(see/seeing/saw).",
},
{
    "question_en": "She avoided ____ him after the argument.
(confront/confronting/confronted)",
    "options_en": [
        "confront",
        "confronting",
        "confronted"
    ],
    "student_answer_en": "She avoided **_confronting_**
(confront/confronting/confronted).",
},
{
    "question_en": "They were accused of ____ confidential information.
(leak/leaking/leaked)",
    "options_en": [
        "leak",
        "leaking",
        "leaked"
    ],
    "student_answer_en": "They were accused of **_leaking_**
(leak/leaking/leaked).",
},
{
    "question_en": "One thousand people of eighteen years and over were
questioned in my town about their ways of ____ . (take/taking/taken)",
    "options_en": [
        "take",
        "taking",
        "taken"
    ],
    "student_answer_en": "One thousand people of eighteen years and over were
questioned in my town about their ways of **_taking_** (take/taking/taken).",
},
{

```

```

        "question_en": "He is passionate about ____ environmental change.
(drive/driving/drove)",
        "options_en": [
            "drive",
            "driving",
            "drove"
        ],
        "student_answer_en": "He is passionate about **_driving_**
(drive/driving/drove).",
    },
    {
        "question_en": "She gave up ____ sweets to improve her health.
(eat/eating/ate)",
        "options_en": [
            "eat",
            "eating",
            "ate"
        ],
        "student_answer_en": "She gave up **_eating_** (eat/eating/ate).",
    },
    {
        "question_en": "They succeeded in ____ the project on time.
(complete/completing/completed)",
        "options_en": [
            "complete",
            "completing",
            "completed"
        ],
        "student_answer_en": "They succeeded in **_completing_**
(complete/completing/completed).",
    }
],
"word_bank": null
}
}
}

```